PROJETO AVALIATIVO · MÓDULO 2 · SEMANA 13

Churn Prediction API

Machine Learning + FastAPI + Streamlit para prever quais clientes vão cancelar o contrato — antes que cancelem.

Jorge Luiz Susin Rodrigues
Análise Preditiva de Dados & IA · SCTEC SC
github.com/JorgeRodrigues2026 · Outubro de 2026

Python | scikit-learn | FastAPI | Pydantic | Streamlit | Plotly | pytest

O PROBLEMA

Perder um cliente custa mais caro do que mantê-lo

Uma operadora de telecom precisa saber **quem está prestes a cancelar** para agir a tempo — com desconto, mudança de plano ou suporte. Sem isso, a retenção é feita às cegas ou depois que o cliente já saiu.

Objetivo do sistema

- Receber os dados de um cliente via **API REST**
- Devolver a **probabilidade de churn** e uma **faixa de risco** (baixo, médio, alto)
- Oferecer um **painel visual** para o time de retenção priorizar contatos

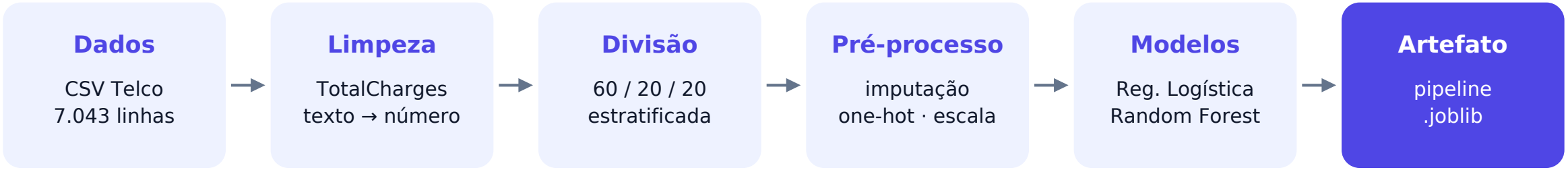
7.043
clientes na base IBM Telco

26,5%
cancelaram (classe minoritária)

19
características por cliente

FASE 1 · PREPARAÇÃO DO MODELO

Pipeline de dados e treinamento



Decisões técnicas

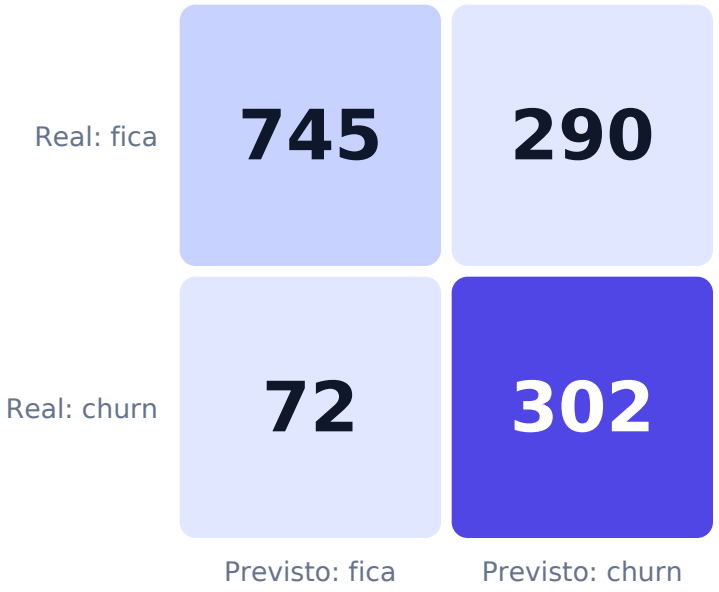
- **Imputação dentro do pipeline**, depois da divisão — evita vazamento de dados do teste para o treino.
- **OneHotEncoder(handle_unknown="ignore")** — uma categoria nova em produção não derruba a API.
- **class_weight="balanced"** — compensa o desbalanceamento (só 26,5% de churn).
- **Validação escolhe, teste mede:** o conjunto de teste só é usado uma vez, no final.
- Pré-processamento + modelo salvos **juntos** em um único arquivo: a API recebe dados crus.

FASE 1 · RESULTADOS

Resultados no teste: 1.409 clientes nunca vistos

ROC-AUC	Recall	Precisão	F1-Score	Acurácia
0,832	80,8%	51,0%	0,625	74,3%

Matriz de confusão



Leitura de negócio

- Dos **374** clientes que realmente cancelaram, o modelo encontrou **302** (81%).
- Custo: **290** clientes fiéis recebem uma oferta sem precisar — bem mais barato que perder um cliente.
- **Por que Regressão Logística?** Empatou com o Random Forest na validação (ROC-AUC 0,859 × 0,859), mas é mais simples, leve (~9 KB) e **interpretável**.

Camadas com responsabilidades separadas

Dashboard

Streamlit
api_client.py
(ou cURL, Swagger,
outro sistema)

JSON →

← JSON

HTTP / REST

Lifespan: o disco é lido **uma única vez** no startup. Um teste automatizado faz 5 predições e confirma 1 só chamada a joblib.load.

FastAPI · app/

routers/ — `predict.py · health.py`
recebem a requisição HTTP e devolvem o status code

↓

schemas/ — `churn.py`
Pydantic valida tipos e limites → 422 se inválido

↓

services/ — `model_service.py`
regras de negócio (→ 400) e inferência

↓

modelo em RAM — `churn_pipeline.joblib`
carregado 1 vez no lifespan, guardado em app.state

Endpoints e status codes semânticos

Rota	Função
`GET /health`	Serviço de pé? Modelo na RAM?
`POST /v1/predict`	Predição de 1 cliente
`POST /v1/predict/batch`	Predição de 1 a 100 clientes
`GET /v1/model/info`	Métricas do modelo em produção

Código	Quando acontece
200	Predição feita com sucesso
400	Tipos certos, combinação incoerente
422	Tipo errado, campo faltando ou extra
500	Falha inesperada (tratada, não cai)
503	Modelo não carregado

Exemplo de resposta

```
{
  "churn_prediction": true,
  "churn_probability": 0.8865,
  "risk_level": "alto",
  "model_version": "1.0.0"
}
```

400 × 422 na prática

- `"tenure": "dez"` → **422** — o Pydantic barra antes do modelo.
- Sem internet mas com streaming → **400** — tipos válidos, regra de negócio violada.
- `extra="forbid"` — campo desconhecido no JSON também é 422.

QUALIDADE

Validação comprovada por testes



Laboratório da API no dashboard: 9 de 9 cenários com o status esperado

13

testes automatizados (pytest) passando

9 / 9

cenários REST com o status esperado

1×

leitura do modelo, não importa quantas requisições

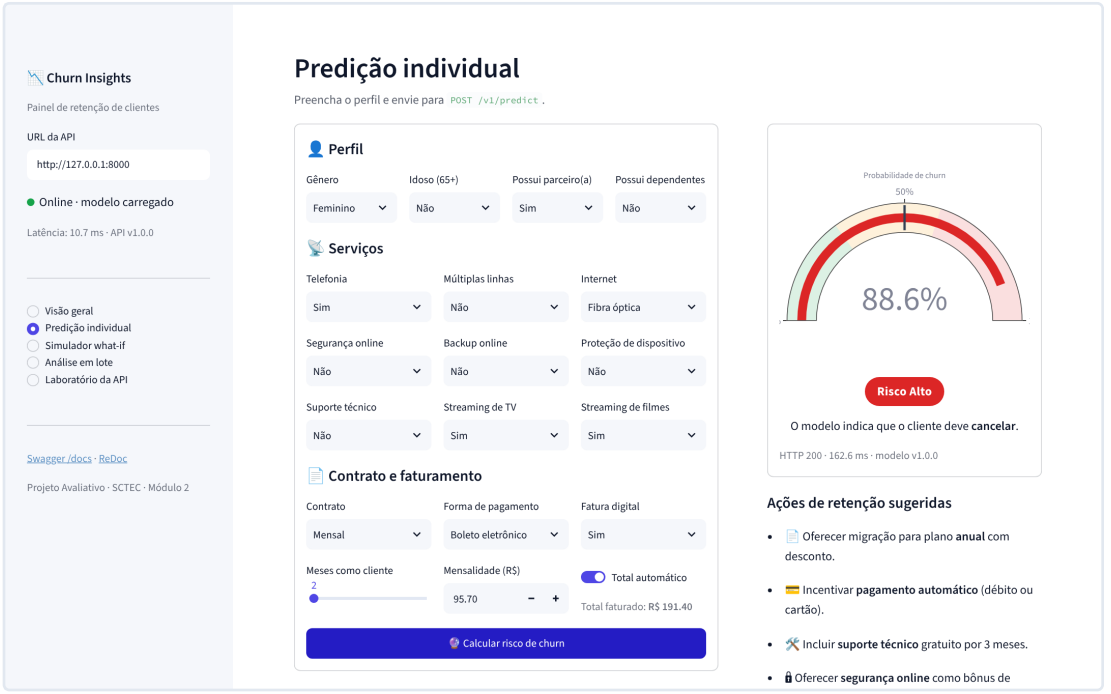
Os testes cobrem 200, 400, 422, 503 e o comportamento do lifespan.

DASHBOARD STREAMLIT

Churn Insights — interface para o time de retenção



Visão geral: saúde da API e métricas do modelo

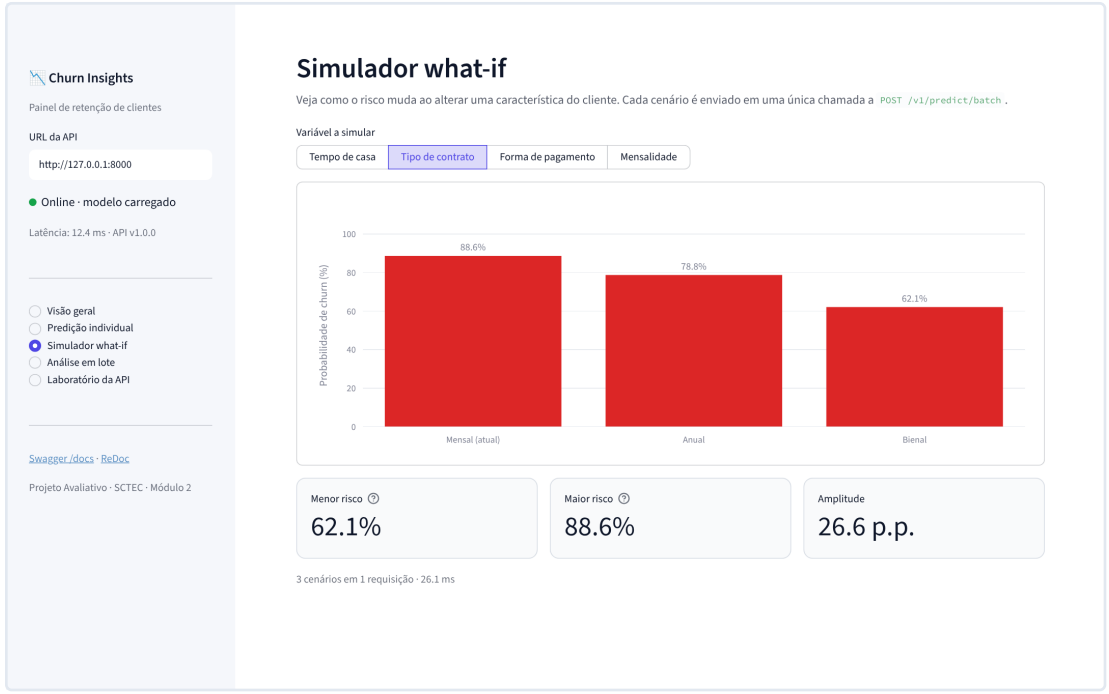


Predição individual: velocímetro de risco + ações sugeridas

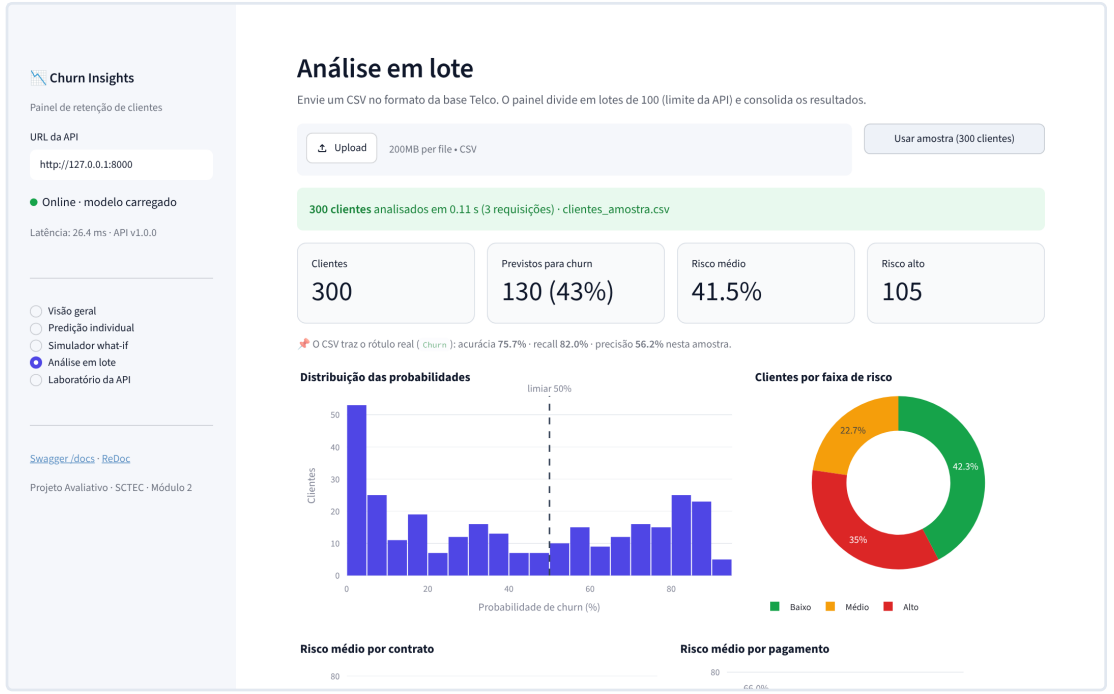
O dashboard **não importa o modelo**: conversa com a API só por HTTP, como qualquer sistema externo.

DASHBOARD STREAMLIT

Simulação de cenários e análise em lote



What-if: como o risco muda ao trocar o tipo de contrato



Lote: 300 clientes em 3 requisições, com ranking de prioridade

Na amostra com o churn real conhecido: **acurácia 75,7%** e **recall 82,0%** — consistente com o resultado do teste.

EXECUÇÃO E ORGANIZAÇÃO

Como rodar e como o código foi versionado

```
# 1. ambiente
python -m venv .venv
pip install -r requirements.txt

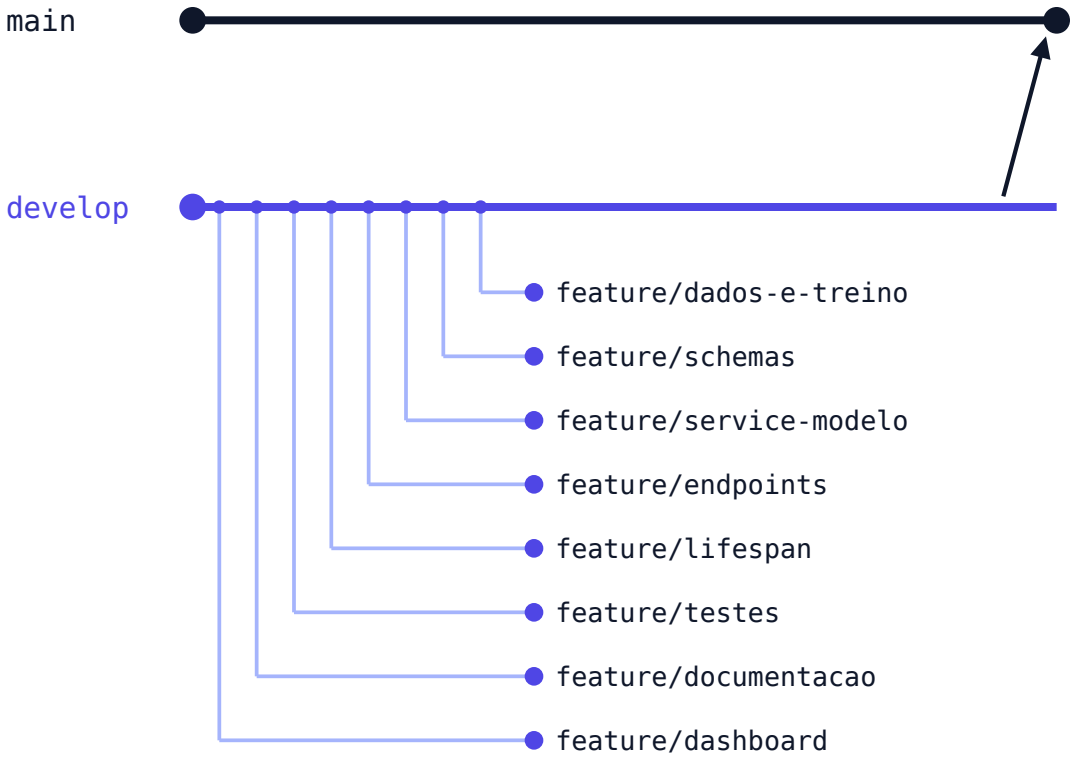
# 2. API  → localhost:8000/docs
uvicorn app.main:app --reload

# 3. dashboard → localhost:8501
streamlit run dashboard/app.py

# 4. testes
pytest -v
```

Treino opcional (o modelo já vem pronto): `python training/train_model.py`

GitFlow



Cada fase em sua branch, com commits pequenos e merge via Pull Request.

CONCLUSÃO

O que aprendi e próximos passos

Aprendizados

- Um modelo só gera valor quando vira um **serviço** que outros sistemas consomem.
- Validar na entrada (Pydantic) é mais barato do que depurar uma predição errada.
- Separar camadas deixa o código **testável** e fácil de trocar o modelo.
- A métrica certa depende do negócio: aqui, **recall** pesa mais que acurácia.

Próximos passos

- Ajustar o limiar de decisão pelo **custo real** de uma ação de retenção.
- Explicar cada predição com **SHAP** ou coeficientes.
- Versionar modelos com **MLflow** e monitorar **drift** dos dados.
- Empacotar em **Docker** e publicar em nuvem.